

Pipeline V3 — Project Report

Date: 2026-06-05 Repository: pipeline-v3 Goal: Ingest Kubernetes workload metrics, engineer features, forecast CPU/memory, estimate risk, and drive resource optimization + governance decisions via decoupled agents.

1) Executive summary

This project implements an agentic, decoupled two-stage pipeline for Kubernetes workload observability and forecasting:

- Stage 1 (Agent 1) — Ingestion: queries Prometheus, engineers and normalizes features, stores them in Redis/InfluxDB/CSV, and emits a durable completion event.
- Stage 2 (Agent 2) — Prediction: waits for ingestion completion events, builds model inputs, runs a PatchTST-based model to forecast CPU/memory, computes throttle/OOM risk, and writes outputs to Redis Streams/InfluxDB/CSV.

Two downstream decision agents consume predictions:

- Agent 3 — Optimization: consumes prediction events and recommends scale/hold/retrain actions (QoS-aware). Publishes optimization decisions.
- Agent 4 — Governance: evaluates optimization decisions using rules and (optionally) an LLM reasoner, then publishes the final governance outcome.

The system's key design choice is loose coupling: agents communicate through shared state + streams, not direct method calls.

2) Architecture overview

2.1 Components

- Runtime/CLI: `src/agents/runtime.py`
- Agents 1 & 2:
- Ingestion agent: `src/agents/ingestion_agent.py`
- Prediction agent: `src/agents/prediction_agent.py`
- Decision agents:
- Agent 3 optimization: `decision_agents/agent3/agent3_optimization.py`
- Agent 4 governance: `decision_agents/agent4/agent4_governance.py`
- Storage layers:
- Redis: `src/storage/redis_store.py`
- InfluxDB 2.x: `src/storage/influxdb_store.py`

- CSV store: `src/storage/csv_store.py`
- Prometheus push (optional): `src/storage/prometheus_writer.py`

2.2 Event streams (contracts)

Agents use Redis Streams as durable triggers:

- `stream:ingestion:complete` — written by Agent 1
- `stream:prediction:complete` — written by Agent 2
- `stream:optimization:complete` — written by Agent 3
- `stream:governance:complete` — written by Agent 4

2.3 State stores

- Redis: fast-access “latest features” cache plus streams.
- InfluxDB: time-series persistence for metrics and predictions (also used for context windows / evaluation utilities).
- CSV: append-only audit trail for quick inspection and offline analysis.

3) Dataflow

1. Agent 1 collects metrics from Prometheus (container + kube-state-metrics + node metrics), engineers derived features, and normalizes features for stable inference. 2. Agent 1 writes: - Latest features to Redis - Time-series metrics to InfluxDB - Per-target CSV rows in `data/csv/metrics/` - Emits `stream:ingestion:complete` 3. Agent 2 listens to `stream:ingestion:complete`. 4. Agent 2 builds model inputs using the same feature pipeline metadata as training and runs inference: - CPU forecast and memory forecast at configured horizons/quantiles - Risk estimation (throttle + OOM) combining model outputs and rule-based enrichment 5. Agent 2 writes: - Prediction CSV rows in `data/csv/predictions/` - InfluxDB predictions measurement - `stream:prediction:complete` 6. Agent 3 consumes predictions and recommends an action (scale up/down/hold/retrain). Publishes `stream:optimization:complete`. 7. Agent 4 applies governance (rules + optional LLM escalation) and publishes `stream:governance:complete`.

4) Model and training

4.1 Model

- The canonical model/feature pipeline is defined in `train.py`.
- The runtime loads a checkpoint (`data/models/patchtst_multi.pt`) that contains:
 - feature column ordering
 - normalization statistics (μ/σ)
 - model weights

- metadata used for consistent inference

4.2 Online fine-tuning

- Incremental training is implemented in `src/training/incremental_trainer.py`.
- It fine-tunes from raw CSV metrics (e.g., `__raw.csv`) with guardrails:
- cooldowns
- maximum retrains per day
- predictor hot-reload after updating the checkpoint

5) What we did (this run)

5.1 Environment and setup

- Activated the project's Python virtual environment (`.venv`).
- Used local infrastructure:
- Redis running on `localhost:6380`
- InfluxDB on `http://localhost:8086`
- Used a test Kubernetes cluster with Prometheus exposed at `http://localhost:30000`.

5.2 Started the pipeline

- Ran Agents 1+2 together via module execution (ensuring `PYTHONPATH` includes repo root):
- `PYTHONPATH="$PWD" python -m src.agents.runtime --agent both --namespace test-workload --pod <pod> --container stress-container --prometheus-url http://localhost:30000 --model-path data/models`

5.3 Validated telemetry output

- Observed ingestion writing metric points into InfluxDB (periodic writes).
- Observed prediction output being appended to CSV files under:
- `data/csv/predictions/...`
- Confirmed that the prediction CSV contains structured forecast/risk fields (not raw console logs).

5.4 Operated downstream agents

- Restarted Agents 3 & 4 in Redis stream mode.
- Cleared single-instance locks when needed via Redis key deletion:
- `lock:agent3:optimization`
- `lock:agent4:governance`

6) How to run (recommended)

Option A — Start all four agents (supported launcher)

From repo root:

- `bash scripts/run_all_agents.sh --source prometheus --namespace <ns> --pod <pod> --container <c> --prometheus-url http://localhost:30000 --model-path data/models`

Option B — Run Agents 1+2 only

- `PYTHONPATH="$PWD" ./venv/bin/python -m src.agents.runtime --agent both --namespace <ns> --pod <pod> --container <c> --prometheus-url http://localhost:30000 --model-path data/models`

Option C — Run Agents 3+4 only

- Agent 3:
- `PYTHONPATH="$PWD" ./venv/bin/python decision_agents/agent3/agent3_optimization.py --mode redis --redis-host localhost --redis-port 6380 --log-file decision_agents/agent3/agent3_log.txt`
- Agent 4:
- `PYTHONPATH="$PWD" ./venv/bin/python decision_agents/agent4/agent4_governance.py --mode redis --redis-host localhost --redis-port 6380 --log-file decision_agents/agent4/agent4_log.txt`

7) Outputs and where to look

- Metrics CSV: `data/csv/metrics/` (includes `__raw.csv`)
- Predictions CSV: `data/csv/predictions/`
- Agent 3 logs: `decision_agents/agent3/agent3_log.txt`
- Agent 4 logs: `decision_agents/agent4/agent4_log.txt`
- InfluxDB:
- measurement metrics
- measurement predictions

8) Known operational footnotes

- The pipeline-agentic console script can fail with `ModuleNotFoundError: No module named 'src'` if the package isn't installed editable and `PYTHONPATH` isn't set. Running `python -m src.agents.runtime` with `PYTHONPATH="$PWD"` avoids this.
- Decision Agents 3/4 use Redis locks to enforce single-instance behavior; stale locks can block restarts.